Zonoth

SESSION 07 · 2H

Networking: Services, DNS & Ingress

How a Pod that changes IP every rollout still gets a stable address — and how the outside world reaches Pinboard.

Pod networking | Services & DNS | Ingress | Gateway API | Lab 07

SESSION 07

What we'll cover today

1 · The Pod network

One IP per Pod, a flat network, CNI — and why that alone is not enough to call an application.

2 · Services & DNS

ClusterIP, NodePort, LoadBalancer, ExternalName, headless; EndpointSlices, kube-proxy and CoreDNS names.

3 · Ingress & policy

HTTP routing with ingress-nginx on kind, TLS in one slide, Gateway API as the successor, NetworkPolicy.

4 · Lab 07

Expose Pinboard: Services for api and web, a DNS test from a debug Pod, then an Ingress on `http://localhost`.

01

The Pod network model

Every Pod gets a real IP on a flat network — and loses it the moment it is replaced.

THE RULES

One IP per Pod, no NAT

An IP per Pod

Not per container and not per node. All containers in a Pod share one network namespace and talk over `localhost`.

A flat network

Every Pod reaches every other Pod on any node **directly**, without NAT and without port mapping.

Nodes are in it too

Agents on a node (kubelet, kube-proxy) can reach the Pods on it — that is how probes work.

The IP is disposable

Rescheduled, restarted, scaled or rolled out → new Pod, new IP. Never hard-code one.

`kubectl get pods -o wide` shows the Pod IPs. Watch them all change during a `rollout restart`.

WHO IMPLEMENTS IT

CNI: the plugin that wires Pods up

Kubernetes defines the **contract**; a CNI plugin makes it true on your infrastructure.

- Assigns Pod IPs (IPAM) and creates the veth pair into the Pod
- Programs routes / overlay so cross-node traffic works
- Usually also **enforces NetworkPolicy** — not all plugins do

Common plugins

kindnet (our kind cluster) · **Calico** · **Cilium** (eBPF) · **Flannel** · cloud-native ones: AWS VPC CNI, Azure CNI, GKE.

⚠ Pick the CNI when you build the cluster. Swapping it later means re-IP-ing every Pod.

THE PROBLEM

Why we need Services

▲ Talking to Pods directly

- 3 replicas = 3 IPs to discover and track
- Every rollout invalidates all of them
- No load balancing, no health awareness
- Web would need to watch the API server

✓ Talking to a Service

- One stable virtual IP and one DNS name
- Membership follows a **label selector**
- Only **ready** Pods receive traffic
- Traffic is spread across the replicas

🔗 A Service is the stable front door for a set of Pods: `pinboard-web` → `http://pinboard-api:8080`, forever.

THE OBJECT

Anatomy of a Service

- `selector` — which Pods, by label, in this namespace
- `port` — what callers dial on the Service
- `targetPort` — the container port; a **named** port survives renumbering
- `type` defaults to `ClusterIP`

> Service and Deployment are independent objects — the selector is the only link.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: pinboard-api
  namespace: pinboard
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: pinboard-api
  ports:
    - name: http
      port: 8080
      targetPort: http
```

UNDER THE HOOD

From selector to packets

01 — Selector
Service names labels, not Pods

→

02 — EndpointSlice
Controller lists the **ready** Pod IPs in `discovery.k8s.io/v1` objects

→

03 — kube-proxy
Programs nftables / iptables / IPVS rules on every node

→

04 — DNAT
ClusterIP:port is rewritten to one Pod IP:targetPort

⚠ EndpointSlices (max 100 endpoints each) replaced the old `Endpoints` object, which is deprecated since Kubernetes 1.33. Read `kubectl get endpointslices`, not `get endpoints`.

LIVE DEMO

One Service in front of three API Pods

Create the ClusterIP, watch the EndpointSlice follow `kubectl scale`, and curl the Service from inside the cluster — hostnames rotate.

What you'll see · endpoints appear and disappear live **Files** · demos/session07/svc-clusterip.yaml **Duration** · 6 min

02

Service types & cluster DNS

Five ways to publish a Service — and the names CoreDNS gives them.

CHOOSE ONE

The five Service flavours

ClusterIP

Default. A virtual IP reachable **only inside** the cluster. Everything Pinboard does internally.

NodePort

Same, plus a port `30000-32767` open on **every node**. Blunt, but works anywhere.

LoadBalancer

NodePort plus a real cloud load balancer, provisioned by the cloud controller manager.

ExternalName

No proxying at all — CoreDNS returns a `CNAME` to an external host.

Headless

`clusterIP: None`. DNS returns the **Pod IPs** themselves. Used by StatefulSets.

Rule of thumb

ClusterIP for everything, and **one** Ingress or Gateway at the edge.

port vs targetPort vs nodePort

- `port` — on the Service's virtual IP
- `targetPort` — on the **Pod**; number or port name
- `nodePort` — on every node's real IP (NodePort/LoadBalancer only)

> Leave `nodePort` empty and let Kubernetes allocate it — hard-coded ports collide across teams.

YAML

```yaml
spec:
type: NodePort
selector:
  app.kubernetes.io/name: pinboard-web
ports:
  - name: http
    port: 8080          # ClusterIP:8080
    targetPort: http    # container port 8080
    nodePort: 30080     # node:30080
```

Callers inside: `pinboard-web:8080`. From your laptop with kind: `localhost:30080` if mapped.

REALITY CHECK

NodePort and LoadBalancer in practice

On a managed cluster

`type: LoadBalancer` makes the cloud controller manager create an AWS NLB / Azure LB / GCP forwarding rule and fill in `status.loadBalancer`. One public IP — and one bill — per Service.

On kind

There is no cloud, so a LoadBalancer Service stays `<pending>` forever. Options: NodePort, `kubectl port-forward`, MetalLB / cloud-provider-kind, or — what we do — **an Ingress on the mapped ports 80/443**.

⚠ Never expose dozens of NodePorts to users: no TLS, no host or path routing, ugly ports, and every node is a target.

Headless and ExternalName

Headless (`clusterIP: None`): no virtual IP, no kube-proxy rules. DNS returns one A record per ready Pod — the client decides.

- Stable per-Pod names for StatefulSets (session 08)
- Clients that do their own pooling: databases, gRPC, Kafka

ExternalName: a DNS alias to something outside the cluster — swap a managed database for an in-cluster one without touching the app.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: pinboard-db
  namespace: pinboard
spec:
  clusterIP: None          # headless
  selector:
    app.kubernetes.io/name: pinboard-db
  ports:
    - name: postgres
      port: 5432
      targetPort: postgres
```

SERVICE DISCOVERY

Cluster DNS: the names you actually type

- CoreDNS runs in `kube-system`; kubelet points every Pod at it
- Same namespace → the short name is enough
- Cross-namespace → add the namespace
- Headless → A records for the Pods; `SRV` records for named ports

```text
 pinboard-api
pinboard-api.pinboard
pinboard-api.pinboard.svc
pinboard-api.pinboard.svc.cluster.local

# a headless Pod, by ordinal
pinboard-db-0.pinboard-db.pinboard.svc.cluster.local

# <service>.<namespace>.svc.<cluster-domain>
```

✓ Pinboard web is configured with `API_URL=http://pinboard-api:8080` — nothing else.

LIVE DEMO

DNS from a debug Pod, and a NodePort on kind

Resolve the Service by short name and by FQDN, look at the resolver config, then reach a NodePort from the host.

What you'll see · CoreDNS answers, search domains, node ports **Files** · none — all imperative **Duration** · 6 min

03

Ingress, Gateway API & NetworkPolicy

One entry point for HTTP — and a first rule about who may talk to whom.

THE GAP

Why Services are not enough at the edge

A Service is **layer 4**. It cannot read a URL.

- No host routing (`api.example.com` vs `www.example.com`)
- No path routing (`/api` vs `/`)
- No TLS termination, no redirects, no rewrites
- One cloud load balancer per Service gets expensive fast



Ingress

One HTTP(S) entry point in front of many Services, with host and path rules — expressed as a normal Kubernetes object.

Pinboard needs exactly this: `/` → web, `/api` → api, on one address.

TWO HALVES

The rule and the thing that enforces it

01 — Ingress object
Your host/path rules. Inert on its own.

→

02 — IngressClass
`ingressClassName` picks which controller owns it

→

03 — Controller
ingress-nginx, Traefik, HAProxy, cloud controllers — Pods that watch the API

→

04 — Proxy config
Reloaded live; traffic goes straight to Pod IPs

⚠ No controller installed → your Ingress is accepted, gets no address, and nothing happens. Always install one first.

MANIFEST

Pinboard's Ingress

- `networking.k8s.io/v1` — the only version since 1.22
- `ingressClassName: nginx`, never the old annotation
- `pathType`: `Prefix`, `Exact` or `ImplementationSpecific`
- The second rule sends `/` (Prefix) to `pinboard-web` port `http`
- Longest path wins, so `/api` beats `/`

YAML

```yaml
  apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: pinboard
  namespace: pinboard
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: pinboard-api, port: { name: http } }
```

ON OUR CLUSTER

ingress-nginx on kind

```bash
  # the kind node already publishes 80/443 and carries ingress-ready=true
NGINX=https://raw.githubusercontent.com/kubernetes/ingress-nginx
kubectl apply -f $NGINX/main/deploy/static/provider/kind/deploy.yaml

kubectl -n ingress-nginx wait --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=120s

kubectl -n pinboard apply -f 40-ingress.yaml
curl -s http://localhost/api/info
```

⚠ ingress-nginx is in maintenance and the project has announced its retirement — for new platforms in 2026, start on **Gateway API**. It is still the best thing to learn Ingress on.

ONE SLIDE ONLY

TLS termination

- The certificate lives in a `kubernetes.io/tls` Secret (`tls.crt` + `tls.key`)
- The controller terminates TLS; traffic to Pods is plain HTTP inside the cluster
- **cert-manager** issues and renews Let's Encrypt certificates automatically
- For mTLS between Pods you want a service mesh, not an Ingress

YAML

```yaml
spec:
ingressClassName: nginx
tls:
  - hosts: ["pinboard.example.com"]
    secretName: pinboard-tls
rules:
  - host: pinboard.example.com
    http:
      paths: [ ... ]
```

In the lab we stay on plain HTTP over `localhost`.

THE SUCCESSOR

Gateway API: role-oriented routing

GatewayClass

Installed by the infrastructure provider — "this kind of gateway exists".

Gateway

The cluster operator's listener: addresses, ports, TLS, which routes may attach.

HTTPRoute

The app team's rules: matches, header/weight based splitting, filters, backends.

- GA since Gateway API v1.0 — `gateway.networking.k8s.io/v1`, installed as CRDs
- Native traffic splitting, header matching, rewrites — no annotation soup
- `GRPCRoute`, `TCPRoute` and cross-namespace attachment via `ReferenceGrant`

SAME ROUTING, NEW API

Pinboard as an HTTPRoute

- `parentRefs` attaches the route to a Gateway
- `matches` replaces `pathType`
- `backendRefs` points straight at Services — add `weight` for canaries
- A second route for `/` → `pinboard-web` completes it

> Ingress keeps working. Migrate route by route, not cluster by cluster.

YAML

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: pinboard
  namespace: pinboard
spec:
  parentRefs:
    - name: pinboard-gw
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      backendRefs:
        - name: pinboard-api
          port: 8080
```

LIVE DEMO

Pinboard in a browser, through the Ingress

Install ingress-nginx on kind, apply the Ingress, open `http://localhost` — then peek at the same rule expressed as an HTTPRoute.

What you'll see · path routing, controller logs, 404 vs 503

Files · demos/session07/ingress.yaml · httproute.yaml

Duration · 8 min

WHO MAY TALK TO WHOM

NetworkPolicy: the default is allow-all

Flat means flat

Without policies any Pod in the cluster can reach your database Pod on 5432. Namespaces are not a boundary.

Selecting = denying

The moment one policy selects a Pod for a direction, everything not explicitly allowed in that direction is dropped.

Additive & namespaced

Policies union; there is no "deny" rule and no ordering. Write allow-lists per workload.

The CNI enforces it

No policy-capable plugin → the objects are stored and ignored. Always test that a denied call really fails.

MANIFEST

Only the API may reach the database

- `podSelector` — the Pods this policy protects
- `policyTypes` — which directions it governs
- `from` — `podSelector`, `namespaceSelector` or `ipBlock`
- The companion policy lets only web and ingress-nginx reach the API on 8080

Result: web → api → db, and nothing else.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: db-only-from-api, namespace: pinboard }
spec:
  podSelector:
    matchLabels: { app.kubernetes.io/name: pinboard-db }
  policyTypes: ["Ingress"]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app.kubernetes.io/name: pinboard-api }
    ports:
    - port: 5432
```

LAB 07 · 30–40 MIN

Expose Pinboard

Give the API a ClusterIP, prove DNS works from a throwaway Pod, deploy the web tier pointing at `http://pinboard-api:8080`, install ingress-nginx on kind and route `/` and `/api` — then open the app in your browser for the first time.

You'll practise: ClusterIP Services · `nslookup` from a debug Pod · EndpointSlices · ingress-nginx on kind · path routing · troubleshooting a 503

Key takeaways

✓ Pod IPs are disposable

Flat network, one IP per Pod, provided by the CNI — never an address you write down.

✓ A Service is a name + a selector

EndpointSlices track the ready Pods; kube-proxy does the DNAT; CoreDNS gives you `svc.ns.svc.cluster.local`.

✓ One edge, not many

ClusterIP inside, one Ingress (or Gateway) outside. Gateway API is where the API is going.

✓ Next up

Session 08 — ConfigMaps, Secrets, volumes and a real database with a StatefulSet.

Zonoth

Questions?

Up next — Session 08: Configuration & storage. Keep the cluster running, we add the database next.